

Recording, analysis and visualization of data from Advacam miniPIX (EDU) silicon pixel detector

Günter Quast, Aug. 2025, last updated June 2026

mPIXdaq Data acquisition and analysis for *miniPIX (EDU)* pixel detector

Vers. 1.1.1, June 2026

DOI [10.5281/zenodo.19280859](https://doi.org/10.5281/zenodo.19280859)

The miniPIX EDU is a camera for radiation based on the Timepix chip with 256x256 radiation-sensitive pixels of 55x55 μm^2 area and 300 μm depth each. The chip is covered by a very thin foil to permit α and β radiation to reach the pixels. The device is enclosed in an aluminum housing with a USB 2.0 interface. The sensor chip is covered with a thin foil and is very fragile; this area should be protected with a cover if not measuring α radiation.

The device provides two-dimensional images of particle traces in the sensitive detector material. The high spatial resolution compared to the typical range of particles in silicon is useful to distinguish the different types of radiation and to measure their deposited energies. α -particles are completely absorbed and deposit all of their energy in the sensitive area, allowing usage of the device as an α spectrometer.

The vendor provides a ready-to-use program for different computer platforms as well as a software-development kit for own applications.

The code provided here serves for acquisition and real-time visualization of data provided by a *miniPIX* or *miniPIX EDU* device in frame mode, i.e. read-out of successive single frames consisting of a full set of 256x256 pixel energies accumulated over a configurable exposure time. Collected frames are displayed as images with a logarithmic color scale representing the deposited energies in each pixel. Real-time clustering of pixels and fast analysis of the cluster properties by means of standard open-source tools for data analysis are also performed. Raw data and analysis results can be exported to files. A sample analysis based on these output data is provided in the form of a Jupyter notebook.

The *mPIXdaq* package is therefore well-suited for high-school or university students to gain detailed insights into the principles of the interaction of different types of radiation with matter and to carry out their own studies by analyzing data recorded during lab courses.

Getting ready for data taking - package installation

This code has been tested on *Ubuntu*, *openSuse*, *Fedora*, on Windows 64bit with *Python3.12.x* and on Raspberry Pi* for the 64-bit versions of *OS12* and *OS13*. Other Linux distributions should not pose any unsurmountable problems.

On MS Windows, the libraries provided by the vendor only supports *Python* vers. 3.12, which is easiest to set up with the *miniconda* framework.

The code also supports devices other than the miniPIX EDU if the vendor-provided calibration file is available.

To get started, follow the steps below:

- Get the code from github

```
git clone https://github.com/GuenterQuast/mPIXdaq or
```

```
git clone https://gitlab.kit.edu/Guenter.Quast/mPIXdaq
```

This repository includes the *Python* code and a minimal set of libraries provided by Advacam.

- Next, cd to the mPIXdaq directory you just downloaded.
- On Linux systems, set up the USB interface of your computer to recognize the miniPIX detector:

```
sudo install_driver_rules.sh
```

 (to be done only once), then connect the *miniPIX* to your computer.

The package may also be installed in your virtual python environment:

- `python -m pip install -r requirements.txt`
- `python -m pip install .`

Note that the *pypixet* initialization is set up use the directory `/var/tmp/mPIX/` resp. `C:\tmp\mPIX\` on MS Windows systems for log-files and configuration. Make sure this directory exist and that you have write access. Put the hardware configuration *xml*-file that came with your device (*MiniPIX-xxx-Wyyyy.xml*) in the `configs/` subdirectory of that location. You may also want to create a subdirectory `factory/` containing the hardware configuration files of all your devices.

Now everything is set up to enjoy your *miniPIX*. Just run the *Python* program from any working directory by typing

```
python run_mPIXdaq.py.
```

If you plan to record data, note that the path to the output file is relative to the current working directory.

It is also worth mentioning that on some systems the current directory, `."`, needs to be contained in the `LD_LIBRARY_PATH` so that the *Advacam Python* interface *pypixet* finds all its *C* libraries. This is also done in the *Python* script `run_mPIXdaq.py` by temporarily modifying the environment variable `LD_LIBRARY_PATH` if necessary and then restarting to execute the *Python* code in the new environment.

Running *mpixDAQ*

Available options of the *Python* program *mPIXdaq.py* to steer data taking, online analysis and and data archival to disk are shown by typing

`run_mPIXdaq.py --help`, resulting in the following output:

```
> run_mPIXdaq.py -h
usage: run_mPIXdaq.py [-h] [-v VERBOSITY] [-o OVERLAY] [-a ACQ_TIME] [-c ACQ_COUNT] [-f FILE] [-w WRITEFILE] [-t TIME]
                        [--circularity_cut CIRCULARITY_CUT] [--flatness_cut FLATNESS_CUT] [-p PRESCALE] [-r READFILE] [-b BADPIXELS]
                        [--no-kbdControl] [--no-guiControl]
```

Read, analyze & display *miniPIX* data

options:

```
-h, --help                show this help message and exit
-v VERBOSITY, --verbosity VERBOSITY
                           verbosity level (1)
-o OVERLAY, --overlay OVERLAY
                           number of frames to overlay in graph (4)
-a ACQ_TIME, --acq_time ACQ_TIME
                           acquisition time/frame in seconds (0.5)
-c ACQ_COUNT, --acq_count ACQ_COUNT
                           frame count for readout via callback (20)
-f FILE, --file FILE      file to store frame data
-w WRITEFILE, --writefile WRITEFILE
                           file to write cluster data
-t TIME, --time TIME      run time in seconds (36000)
--circularity_cut CIRCULARITY_CUT
                           cut on circularity for alpha detection (0.5)
--flatness_cut FLATNESS_CUT
                           cut on flatness for alpha detection (0.6)
-p PRESCALE, --prescale PRESCALE
                           prescaling factor for frame analysis
-r READFILE, --readfile READFILE
                           file to read frame data
-b BADPIXELS, --badpixels BADPIXELS
                           file with bad pixels to mask
--no-kbdControl            switch off keyboard control
```

```
--no-guiControl      switch off gui control
```

As an alternative to the command line, a script `grun_mPIXdaq.py` is provided, which starts a graphical interface for argument setting. This script is particularly useful if tied to a desktop icon.

The default values are adjusted to situations with low rates, where frames from the *miniPIX* with an exposure time of `acq_time = 0.5 s` are read. For the graphics display, `overlay = 4` recent frames are overlaid, leading to a total integration time of 2 s.

The *miniPIX EDU* version, in particular, may suffer from a large number of dead or noisy pixels, and therefore they should be masked by providing a file with indices of pixels to be ignored. The default file name is *Xnn-Wnnnn_badpixels.txt* in the working directory, where Xnn-Wnnnn is unique chip-id of the miniPIX sensor. Alternatively a file name may be specified using the `-b` or `--badpixels` option.

Collected frame data may be directly written to disk, if a filename is given using the `-f` or `--file` option. Two formats are foreseen at present, storage of the two-dimensional frames as *numpy*-arrays (file extension `.npz`) or as lists of pixel indices and energy values in *yaml*-format (file extension `.yaml`). If no suffix for the filename is given, the default behavior is writing a `.yaml` file. The `.yaml` format also permits storing meta data like parameters of the data acquisition, the properties of the sensor and the list of bad pixels. To save space, the output files may be compressed with *zip* or *gzip*.

The same formats are recognized when reading back files using the `-r` resp. `--readfile` options. In addition, files written with the *Pixet* program of Advacam in *.clog* format can be used as an input to this software package.

Data analysis consists of clustering of pixels in each frame and determination of cluster parameters, like the number of pixels, energy of clusters, a rectangular bounding boxes of each cluster and the shapes of the cluster areas and of the energy distribution over the pixels in the clusters. A detailed description of the stored cluster features and of the algorithms to determine them are given in the *EducatorsGuide* that is part of this package.

Properties of clusters, including a list of contributing pixels and their energy values, are optionally written to a file in *yaml* format (file extension `.yaml`) for later off-line analysis. A more compact version in `.csv` format, containing just the cluster properties, is also available.

A *Jupyter* notebook, *analyze_mPIXclusters.ipynb*, illustrates an example analysis based on these file formats.

To test the software without access to a *miniPIX* device or without a radioactive source, a file with recorded data is provided. Use the option `--readfile data/BlackForestStone.yaml.gz` to start a demonstration. Note that the analysis of the recorded pixel frames is done in real time and may take some time on slow computers. Cluster data in *yaml* format may also be used with the `-r` resp. `--readfile` option for play-back of recorded data with *mPIXdaq*.

Parameter settings for data acquisition

The optimal choice of parameters, in particular the values of exposure time and number of overlaid frames for the graphical display, depend very much on the use case.

In scenarios with low rates aiming at a demonstration of the capabilities of a modern particle detector like the *miniPIX* for outreach purposes, it is most useful to mimic the behavior of a cloud chamber, i.e. particle traces appear, remain visible for some time and then disappear again. This can be achieved by setting an acquisition time of 0.2 s and an overlay of 10 frames. Particles tracks then remain visible on screen for 2 s. The command to run in this mode is:

```
run_mPIXdaq -a0.2 -o10
```

If the goal is to efficiently record particle tracks with low read-out dead-time, it is not necessary to optimize the visual impression, because the graphical display is only used to ensure the quality of the recorded data. To reduce processing overheads, the fraction of events being analyzed and displayed may be pre-scaled. In the example below, only every fourth frame is analyzed and displayed:

```
run_mPIXdaq -a0.2 -p4 -o1
```

In high-rate scenarios exceeding 100 particles/s the read-out efficiency for the miniPIX becomes a concern. The USB 2 transfer and initialization overheads take about 25 ms per frame. In practice, this means that about 20 frames/s of 25 ms exposure time each can be transferred at a read-out dead-time of 50%. The number of objects per frame should

mPIXdaq starter

Read, analyze & display miniPIX data

--verbosity
verbosity level (1)

--overlay
number of frames to overlay in graph (4)

--acq_time
acquisition time/frame in seconds (0.5)

--acq_count
frame count for readout via callback (20)

--file
file to store frame data

--writefile
file to write cluster data

--time
run time in seconds (36000)

--circularity_cut
cut on circularity for alpha detection (0.5)

--flatness_cut
cut on flatness for alpha detection (0.6)

--prescale
prescaling factor for frame analysis

--readfile
file to read frame data

--badpixels
file with bad pixels to mask

--no-kbdControl ☐
switch off keyboard control

--no-guiControl ☐
switch off gui control

Befehl anzeigen Beenden Ausführen

Abbildung 1: Graphical interface to start mPIXdaq

not exceed about 100 in order to avoid overlapping clusters. So, in practice, signatures of 2000 particles/s can be handled, which is clearly sufficient for most laboratory experiments with rather weak radioactive sources that comply with radiation protection regulations.

Read-out of the miniPIX is fastest in callback mode, when the driver is initialized to call a function for data retrieval whenever a new frame is ready to be transferred. Thus only one initialization step is necessary to receive *acq_count* frames. The exposure time of each frame is given by the value of *acq_time*. To achieve maximum read-out speed for such high-rate scenarios, start data-acquisition with the command:

```
run_mPIXdaq -a 0.025 -c50 -p10 -o1
```

With these settings, only one tenth of the frames is analyzed and displayed; the recorded frame rate is 20 Hz, while the read-out dead-time indeed turns out to be 50% (measured on a Raspberry Pi 5).

If higher read-out rates up to the nominal 40 frames/sec are needed, data can be recorded using the *Pixet* (basic) program delivered by *Advacam* together with the device. Select tracking mode, the required exposure-time per frame and the number of frames from the graphical interface, then press the record button, and after completion save the acquired frames using the *save* button selecting *.clog* as the file format. This format is understood by *mPIXdaq* and can be read in via the *-readfile* option, thus allowing you to perform cluster analysis and writing files in the same formats as implemented in *mPIXdaq*. In case your miniPIX device suffers from noisy pixels, a bad-pixel map can be specified using the *-badpixel* option.

Implementation Details

The default data acquisition is based on the function *doSimpleAcquisition()* from the *Advacam Python* API in callback mode, where *acq_counts* frames with an adjustable accumulation time *acq_time* are read from the miniPIX device successively.

The chosen readout mode is “ToT” (“time over threshold”, *PX_TPXMODE_TOT*). This quantity shows good proportionality to the deposited energy at high signal values, but exhibits a non-linear behavior for very small signals near the detection threshold of the *miniPIX*. There are individual calibration constants for each pixel, which are provided for each device by the vendor and serve to provide the deposited energies per pixel in units of keV. At very high pixel energies exceeding 1500 keV/pixel the energy scale is no longer linear, and in particular measurements of the energies of α -particles become imprecise.

The relevant libraries for device control are provided in directories *advacam_<arch>* for x86_64 Linux, arm32 and arm64 and for Macintosh arm64 and MS Windows architectures. The contents of a typical directory is:

```
__init__.py      # package initialization
pypixet.so       # the Pixet Python interface
minipix.so       # C library for pypixet
pxcore.so        # C library for pypixet
pixet.ini        # initialization file, in same directory as pypixet
pixetVersion.py  # version number of the pypixet library
```

Note that the device-dependent *xml*-file for the hardware configuration must also be supplied. The *mPIX* package expects it to be found in the directory *\var\tmp\mPIX\factory* (or *C:\tmp\mPIX\factory* on windows systems). If more than one device is in use, the hardware configuration files for all devices may be stored in this directory and will be automatically detected. Note that without such an individual hardware configuration file the *miniPIX* will not function properly.

Copyright notice:

The copyright of these libraries is held by *Advacam*. They may also be downloaded from their web page, ADVACAM DWONLOADS, and are provided here for convenience as *Python* packages.

On-line Data Analysis

The on-line analysis is intentionally very straight-forward and based on standard libraries and functions. Clustering of pixels is performed by finding connected regions in the pixel image with *scipy.ndimage.label()*. The shape of the clusters is determined from the ratio of the smaller and the larger one of the two eigenvalues of the covariance matrix calculated from the *x* and *y* coordinates of the pixels in a cluster using *numpy.cov()*. For circular clusters, as typically produced by α radiation, this ratio is close to one, while it is almost zero for the longer traces from β radiation. In a

similar way, also the shape of the energy distribution is considered, which shows a sharp maximum at the center for α particles but is rather flat otherwise.

The figure below shows the graphical display with a pixel image and the typical distributions of the pixel and cluster energies and the number of pixels per cluster. The source used was a weakly radioactive stone (approx 10 Bq/cm²) from the Black Forest containing a small amount of Uranium and its decay products. The pixel map shown in the figure was sampled over a time of five seconds. The histogram in the lower-right corner demonstrates that the cluster types of different kinds of radiation are well separated: α rays in the green band with relatively low numbers of pixels per cluster, electrons (β) as long tracks with large numbers of pixels per cluster and rather low energies. Single pixels not associated to clusters mostly originate from γ rays. Some of the electron tracks with typically low energies also stem from photon interactions in the detector material (via the Compton process).

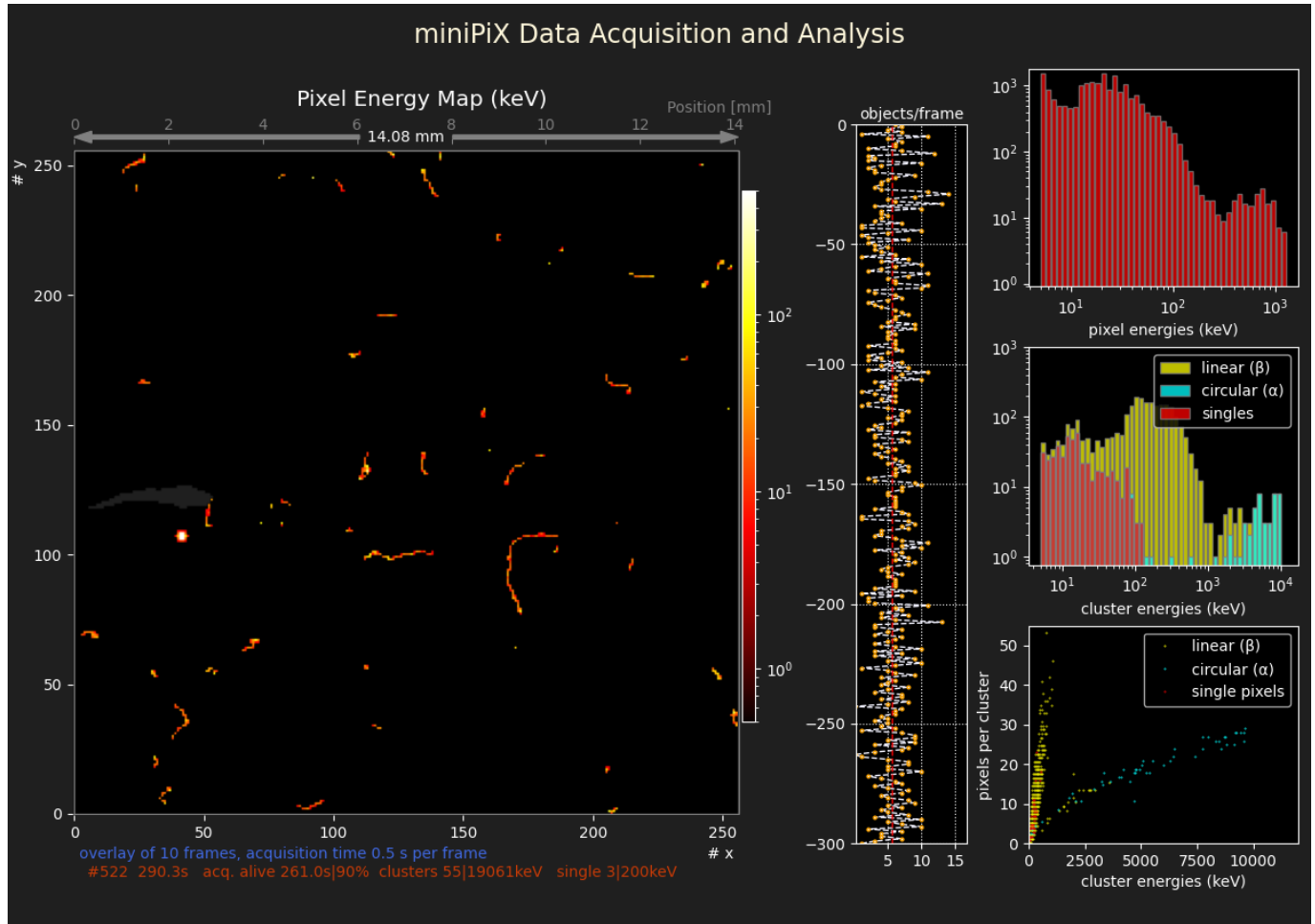


Abbildung 2: The graphical display of miniPIXdaq

The frame collection time is chosen to be on the order of seconds, so that analysis results can be displayed in real-time on a sufficiently fast computer including the Raspberry Pi 5. This is suitable for investigations of natural radiation as emitted by minerals like Pitchblend (=Uraninit), Columbit, Thorianit and others. The detection of Radon and its decay products from the air in basement rooms accumulated on a paper towel with a vacuum cleaner or on the surface of an electrostatically charged balloon also works fine.

For applications at higher rates, the analysis may have to be done off-line by reading data from recorded files. The option `-p` or `-prescale` can be used to limit frame and cluster analysis to a subset of the recorded frames while still allowing to record all data to file with sufficiently low dead time.

In a future version of this program an option to use multiple cores for the analysis task may be provided.

Output files and formats

miniPIXdaq offers the possibility to write raw or clustered data for further off-line analysis, which is a valuable asset for use of the software in physics laboratory courses. This feature enables students to develop their own strategies for the analysis of data recorded in the student lab.

The default output-format is *yaml*, offering human readability, clear structure and modularity of data blocks. Output files are written sequentially as text files while data acquisition is progressing. Files may be compressed using *gzip* or *zip* to obtain more compact representations of the recorded data.

Frame data and **cluster data** including some meta data are stored in *yaml* structures with the keys *meta_data*:, *deviceInfo*:, *bad_pixels*: and *eor_summary*:. Frame or cluster data are stored as lists of pairs of pixel indices and pixel energies for all pixels with non-zero energy under the keys *frame_data*: or *cluster_data*:, respectively. These files can be loaded into a *Python* dictionary using the python code

```
> yaml_dict = yaml.load(open('<filename>', 'r'), Loader=yaml.CLoader)
```

and frame or cluster data unpacked into a *Python* list via

```
> list_of_framedata = yaml_dict["frame_data"] or  
> list_of_clusterdata = yaml_dict["cluster_data"], respectively.
```

Frame data in text form or as *zipped* or *gzipped* files can be used as input to *mPIXdaq* via the '-r' or '-readfile' options.

list_of_clusterdata is a list of two lists, the first one containing cluster properties and the second one indices and energies of contributing pixels, i.e.

```
> list_of_clusterproperties[i] = yaml_dict["cluster_data"][i][0] and  
> list_of_clusterpixels[i] = yaml_dict["cluster_data"][i][1]
```

The keys of the variables in *list_of_clusterproperties* are

```
>['time', 'x_mean', 'y_mean', 'n_pix', 'energy', 'e_mx', 'x_mn', 'y_mn', 'w', 'h',  
'var_mx', 'var_mn', 'angle', 'xE_mean', 'yE_mean', 'varE_mx', 'varE_mn']
```

```
time      : time since start of daq when frame was recorded  
x_mean    : mean x-position of cluster (in pixel numbers)  
y_mean    : mean y-position of cluster (in pixel numbers)  
n_pix     : number of pixels in cluster  
energy    : energy of cluster (= sum of pixel energies) in keV  
e_mx      : maximum pixel energy  
x_mn      : minimum x of bounding box containing the cluster  
y_mn      : minimum y of bounding box  
w         : width of bounding box  
h         : height of bounding box  
var_mx    : maximum variance of geometrical cluster shape (in pixels)  
var_mn    : minimum variance of geometrical cluster shape (in pixels)  
angle     : orientation of cluster (0 = along x-axis, pi/2 = along y-axis)  
xE_mean   : mean x of energy distribution (in pixel numbers)  
yE_mean   : mean y of energy distribution (in pixel numbers)  
varE_mx   : maximum variance of energy distribution  
varE_mn   : minimum variance of energy distribution
```

Empty frames with no clusters or single pixels are represented by an entry containing only the time stamp. This is important for scenarios requiring a detailed analysis of particle rates.

It is also possible to store the cluster properties in simple *.csv* format by explicitly specifying the file extension: `run_mPIXdaq <options> -w <name>_clusters.csv`.

Histograms displayed in the on-line graphical display may be saved using the control buttons in the *matplotlib* window.

Analysis of output data

A **Jupyter notebook**, *analysis/analyze_mPIXclusters.ipynb*, is distributed as part of the package and illustrates how to read and interpret cluster data. This notebook and some *Python* code are collected in the subdirectory *analysis/* of the *mPIXdaq* package. This example is meant as a starting point for data analysis by students.

The *Python* program **ClusterSummary.py** in the subdirectory /analysis reads cluster data written with *mPIXdaq*, provides an overview of data and meta-data and counts α , β , γ and μ signatures with pre-defined selection cuts on the cluster features explained above.

The required parameters can be comfortably set using the graphical interface **grun.py** that reads the argument list of a *Python* program and permits interactive parameter setting before executing the script.

An **analysis example** of data taken over two hours from an air sample collected in a basement rooms on a paper towel with a vacuum cleaner is shown in the output and figure below.

```

*==* Contents of file ../data/Radon_clusters.yml.gz
Data written on Sun Apr 12 23:36:34 2026 with device EduMiniPIX TSWB sn: 2987
16313 frames of 0.5s exposure time
47227 clusters -> rate = 5.79 Hz

```

```

*==* cluster features:
Index(['time', 'x_mean', 'y_mean', 'n_pix', 'energy', 'e_mx', 'x_mn', 'y_mn',
'w', 'h', 'var_mx', 'var_mn', 'angle', 'xE_mean', 'yE_mean', 'varE_mx',
'varE_mn', 'pixels', 'Epix_mean', 'circularity', 'flatness',
'straightness'], dtype='object')

```

```

*==* selection cuts
a: flatness < 0.5
   circularity > 0.5
   emx_cut > 400.0 keV
β: size > 4 and not a
γ: not(a or β)

```

```

*==* α, β, γ Statistics:

```

	α	β	γ	
events	6465	19843	20691	
rate (Hz)	0.793	2.43	2.54	
meanE (keV)	6.29e+03	231	74.9	
sigE (keV)	2.78e+03	95.9	53.3	
muons				5 μ

Package Structure

This package consists of several *Python* files with classes providing the base functionality. As mentioned above, it relies on some Advacam libraries for setting-up and reading-out the sensor. Other dependencies are well-known libraries from the *Python* eco-system for data analysis:

- scipy
 - .ndimage.label
- numpy
 - .cov
 - .linalg.eig
- matplotlib
 - .pyplot

The components, classes and scripts of the package are

- mpixdaq main code handling read-out, data analysis, visualization and storage
 - classes:
 - * mpixControl
 - * miniPIXdaq
 - * frameAnalyzer
 - * mpixGraphs
 - * runDAQ

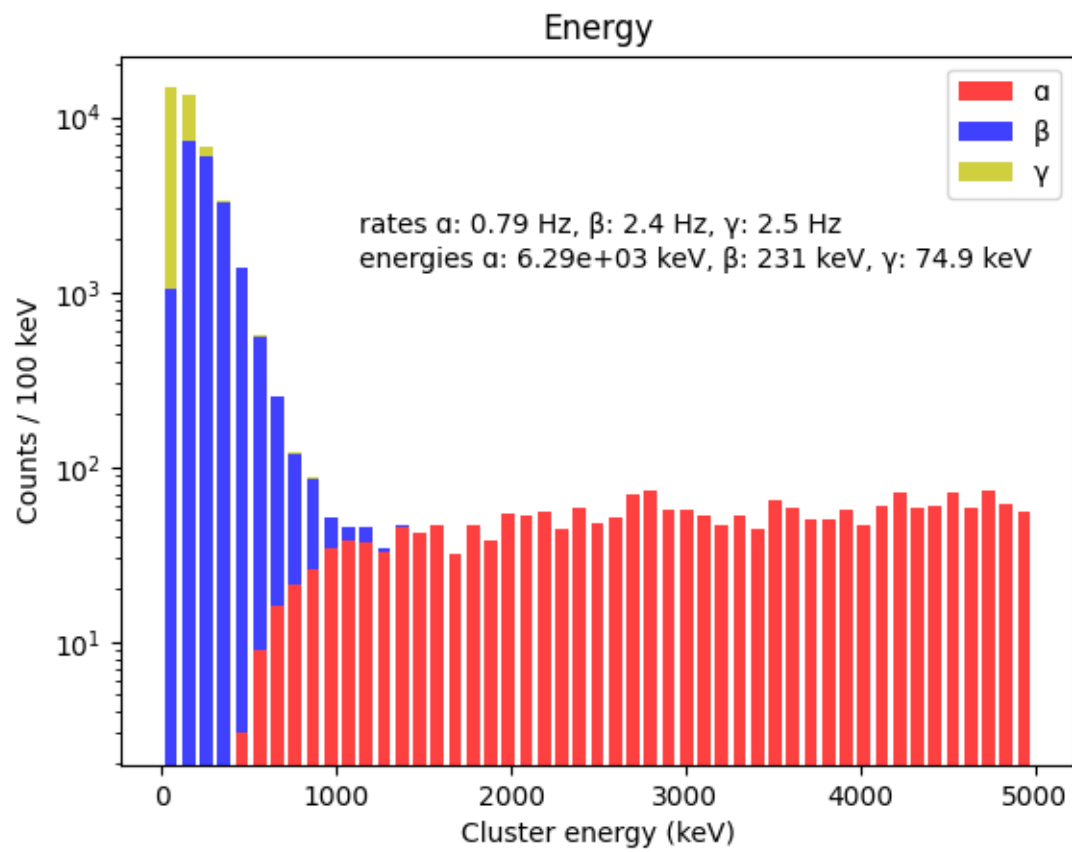


Abbildung 3: Distribution of Cluster energies per particle type, produced by *ClusterSummary.py*

- mplhelpers graphical interface for *mPIXdaq* control with matplotlib
 - class ``bhist`` for animated bar-graph histograms
 - class ``scatterplot`` for animated scatter-plots
 - class ``controlGUI`` for mouse-based control of the data-acquisition process
- mpixelers for decoding supported file formats and plotting of cluster data
 - class `fileDecoders` to decode various input file formats: *mPIXdaq* .npy and .yaml and Advacam .txt and .clog
 - class `clusterReader()` to read (and analyze) pixel clusters written with *mPIXdaq*
 - function `plot_cluster()` to plot energy map of pixel cluster
 - class `shmManager` for management of shared memory blocks
- physics_tools for calculations of the (mean) energy deposits
- package script `run_mPIXdaq.py`
- script `calculate_dEdx` for calculating and plotting the energy loss in matter
- subdirectory `analysis/` with a *Jupyter* notebook to analyze cluster data
 - `analyze_mPIXclusters.ipynb`
 - `ClusterSummary.py` for reading cluster data and providing a summary
 - `LandauFit.py` for fitting a Landau distribution
 - `peakFitter.py` to search for and fit a Gaussian to peaks in spectra
 - `grun.py` is an argparse GUI-Runner to display and set arguments in a GUI and start scrip execution (for those who struggle with the command line)
- subdirectory `data/` contains some small data sets as examples for the *Jupyter* notebook
 - `BlackForestStone.yaml.gz` # *data recorded with a low-activity stone (Uranium ore) from the Black Forest*
 - `BlackForestStone.csv.gz` # *cluster properties of stone from Black Forest*
 - `BlackForestStone_clusters.yaml.gz` # *cluster properties and pixel data of stone from Black Forest*
 - `gammaRadiation_clusters.yaml.gz` # *cluster and pixel data from Black Forest stone shielded with 3mm Aluminium*
 - `ambientRadiation_clusters.yaml.gz` # *cluster and pixel data from long run with ambient radiation*
 - `Radon_clusters.yaml.gz` # *cluster and pixel data from Radon decay products*

Details on the classes and their interfaces are given below.

mpixdaq.py

```
class mpixControl:
    """Provides global variables containing Device Information
    as well as Queues, Events and methods to control threads"""
```

```
class miniPIXdaq:
    """Initialize and readout miniPIX device
```

After initialization, the `__call__()` method of this class is executed in an infinite loop, storing data from the device in a ring buffer. The current buffer index is sent to the calling process via a Queue (`dataQ`, an instance of `threading.Queue()`). The loop ends when the flag `*endEvent*` (an instance of `threading.Event()` in class `mpixControl`) is set.

DAQ parameters from class `mpixControl`:

- `ac_count`: number of frames to read successively
- `ac_time`: acquisition (=exposure) time per frame

Queues for communication and synchronization

- `dataQ`: Queue to transfer data

- real-time analysis of data frames
- animated figures to show a live view of incoming data
- event loop controlling data acquisition, data output to file and graphical display

mpixhelpers.py

```
class fileDecoders:
    """Collection of decoders for various input file formats
    supports mPIXdaq .npy and .yaml and Advacam .txt and .clog
    """

class readClusters:
    """Read cluster data written with mPIXdaq, print meta data and statistics

    Default cuts on cluster features are used to classify clusters as  $\alpha$ ,  $\beta$  or  $\gamma$  signatures.

     $\alpha$ : round cluster shape and peaking energy distribution, high ionization per pixel
     $\beta$ : shape is not round, low ionization per pixel, >5 pixels
     $\gamma$ : not ( $\alpha$  or  $\beta$ )

    Methods:

    * __init__(): instantiate class clusterReader and - optionally - set input file name
    * set_cuts(): set cut values
      - small_cut: separate small and large clusters
      - circularity_cut: round topology
      - flatness_cut: flat energy distribution
      - emean_cut: energy loss per pixel (only used if emx not in feature list)
      - emx_cut: maximum pixel energy
      - no_saturation: ignore clusters with saturated pixels
    * parse_args(): read command line arguments if used interactively
    * read_data(): load data in yaml format in pandas data frame
    * set_selection_masks(): define boolean masks to select  $\alpha$ ,  $\beta$  and  $\gamma$ 
    * get_statistics(): count signatures and provide parameters of energy distributions
    * plot(): plot energy distributions of  $\alpha$ ,  $\beta$  and  $\gamma$ 
    * __call__(): execute read_data(), set_selection_masks() and get_statistics()
    """

def pxl2map(pxlist):
    """Create pixel energy map from pixel list

    Args:
    - pxlist: list of pixels [ ..., [px_idx, px_energy], ...]
    - num: int, for numbering figures

    Returns:
    - numpy array
    """

plot_cluster(pxlist, num=0):
    """Plot energy map of pixel cluster

    Args:
    - pxlist: list of pixels [ ..., [px_idx, px_energy], ...]
    - num: int, for numbering figures

    Returns:
    - matplotlib figure
```

```

"""
class shmManager:
    """simple management of shared memory blocks

    class methods:
        - def get_sharedMem(name, size): create or link to shared memory

        do not forget to close() and finally unlink() all requested blocks
        in calling process
    """

```

mplhelpers.py

```

class bhist:
    """one-dimensional histogram for animation, based on bar graph
    supports multiple data classes as stacked histogram

    Args:
        * data: tuple of arrays to be histogrammed
        * bindegas: array of bin edges
        * xlabel: label for x-axis
        * ylabel: label for y axis
        * yscale: "lin" or "log" scale
        * labels: labels for classes
        * colors: colors corresponding to labels
    """

class scatterplot:
    """two-dimensional scatter plot for animation, based on numpy.histogram2d.
    The code supports multiple classes of data and plots a '.' in the corresponding color in every non-
    zero bin of a 2d-histogram

    Args:
        * data: tuple of pairs of coordinates (([x], [y]), ([], []), ...)
        per class to be shown
        * binedges: 2 arrays of bin edges ([bex], [bey])
        * xlabel: label for x-axis
        * ylabel: label for y axis
        * labels: labels for classes
        * colors: colors corresponding to labels
    """

class controlGUI:
    """graphical user interface to control apps via multiprocessing Queue

    Args:
        - cmdQ: a multiprocessing Queue to accept commands
        - appName: name of app to be controlled
        - statQ: mp Queue to show status data
        - confdict: a configuration dictionary for buttons, format {name: [position 0-
5, command]}
    """

```

The package script `run_mPIXdaq` is provided as an example to tie everything together into a running program. Because the ADVACAM *Python* interface (`pypixet.so`) expects C-libraries and configuration files in the very same directory as the Python interface `pypixet.so` itself, some tricky manipulation of the environment variable `LD_LIBRARY_PATH` is needed to ensure that all libraries are loaded and the *miniPIX* is correctly initialized.

```

#!/usr/bin/env python
#
# script run_mPIXdaq.py
# run mpixdaq example with data acquisition, on-line analysis and visualization
# of pixel frames and histogramming

import os
import platform
import sys
import multiprocessing

# on some Linux systems, pypixet requires '.' in LD_LIBRARY_PATH to find C-libraries
# - add current directory to LD-LIBRARY_PATH
# - and restart python script for changes to take effect

modified_path = False
if platform.system() != 'Windows':
    _ldp = os.environ.get("LD_LIBRARY_PATH")
    if _ldp:
        if '://' not in _ldp and _ldp != '.':
            os.environ["LD_LIBRARY_PATH"] = _ldp + ':'
            modified_path = True
    else:
        os.environ['LD_LIBRARY_PATH'] = '.'
        modified_path = True

# restart script in modified environment
if modified_path:
    print(" ! temporarily added '.' to LD_LIBRARY_PATH !")
    try:
        os.execv(sys.argv[0], sys.argv)
    except Exception as e:
        sys.exit('!!! run_mPIXdaq: Failed to Execute under modified environment: ' + str(e))

# get current working directory (before importing minipix libraries)
wd = os.getcwd()
from mpixdaq import mpixdaq # this may change the working directory, depending on system

# finally, start daq in working directory
if __name__ == '__main__': # -----
    if sys.platform.startswith("win"):
        # On Windows calling this function is necessary.
        multiprocessing.freeze_support()

    rD = mpixdaq.runDAQ(wd)
    rD()

```

It is also possible to start the script as a *Python* module:

```
python -m mpixdaq
```

physics_tool.py contains some code to calculate the energy loss of radiation in matter, based on the Bethe-Bloch formula with corrections for the light electron. Methods to produce graphical output are also included.

```

class materials:
    """Collect properties of target materials and projectiles"""

    dEdx(T, material, projectile):
        """calculate energy loss in matter ( or "mass stopping power")

```

for heavy projectiles and electrons

Parameters:

T: kinetic energy in MeV
Material dictionary
 Z: effective Z
 A: effective A
 I: mean ionization energy (MeV)
z: charge of projectile
m: mass of projectile

Returns:

mass stopping power in units of MeV cm²/g
(to be multiplied by material density to obtain energy loss per unit length)

Formulae:

- Bethe-Bloch relation for heavy projectiles (m_{α} , m_p or m_{μ} $\gg m_e$)
- Notes:
- only collisions considered, no radiation loss (relevant for energy $\gg 1$ MeV)
 - density effect correction (δ) omitted here for simplicity
 - pure Bethe loss decreases below 0.4 MeV and becomes even negative below 0.15 MeV for alpha particles in air; this can be fixed by adding Barkas corrections, which are, however, not implemented here
 - empirical cut-off to avoid negative values of energy loss at small T
- modified Bethe-Bloch equation for electrons, ICRU Report 37 (1984).
"""

calc_pixel_energies(E0, px_size=0.0055):

"""calculate pixel energies for an electron track with energy E0 in silicon

calc_E_vs_depth(E0, dx, material, projectile):

"""calculate particle energies after penetrating depth x of material

plot_dEdx_electron(material, nbins=100, bw=0.05, axis=None):

"""plot dE/dx * rho for electrons as a function of energy

plot_beta_pixel_energies(E0=1.0, px_size=0.0055, axis=None):

"""energy deposits per pixel

E0: initial energy

px_size: pixel size in cm

axis: figure axis; a new one is created if not given

returns: matplotlib figure

"""

plot_dEdx_alpha(material, nbins=200, bw=0.025, axis=None):

"""dE/dx for alpha particles in material vs. energy

material: target material

nbins : number of bins

bw : bin width

axis : figure axis; a new one is created if not given

returns: matplotlib figure

"""

plot_alpha_range(material, E0=6.0, dx=0.050, axis=None):

"""alpha energy after penetration depth in material

material : target

```
E0      : initial energy in MeV
dx      : step width
axis    : figure axis; a new one is created if not given

returns: matplotlib figure
"""
```

A *Python* script `calulate_dEdx.py` illustrates how to use these methods and classes.